

**LAPORAN TUGAS
KECERDASAN BUATAN**

***"Implementasi Algoritma Dijkstra
Untuk Menentukan Jalur Terpendek (Shortest Path)
di Lingkungan Universitas Bengkulu"***



Disusun Oleh :

Lovien Najla Dhafiyah (G1A024055)

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026**

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga saya dapat menyelesaikan Tugas ini dengan baik dan tepat waktu. Tugas ini disusun sebagai salah satu syarat untuk memenuhi penilaian pada mata kuliah Kecerdasan Buatan yang saya tempuh.

Laporan ini membahas proses perancangan dan implementasi aplikasi berbasis web yang berjudul “Implementasi Algoritma Dijkstra Untuk Menentukan Jalur Terpendek (*Shortest Path*) di Lingkungan Universitas Bengkulu.” Proyek ini bertujuan untuk membantu sivitas akademika Universitas Bengkulu dalam menemukan rute perjalanan yang efisien menuju berbagai lokasi di lingkungan kampus dengan mempertimbangkan kondisi operasional gerbang serta alternatif jalur yang tersedia.

Saya mengucapkan terima kasih kepada semua pihak yang telah memberikan dukungan, bantuan, dan bimbingan selama proses pengerjaan tugas ini. Saya menyadari bahwa laporan ini masih memiliki berbagai kekurangan, baik dari segi penulisan maupun pengembangan sistem. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi penyempurnaan karya ini di masa mendatang.

Akhir kata, saya berharap laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai penerapan teknologi *Geographic Information System* (GIS), layanan pemetaan digital, dan pengembangan aplikasi web dalam mendukung sistem navigasi yang cerdas dan efektif di lingkungan kampus.

Bengkulu, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	1
DAFTAR ISI.....	2
DAFTAR GAMBAR	3
BAB 1	5
PENDAHULUAN	5
A. Latar Belakang	5
B. Rumusan Masalah	6
C. Tujuan.....	6
D. Manfaat.....	6
BAB II.....	7
LANDASAN TEORI.....	7
A. Sistem Informasi Geografis.....	7
B. Sistem Navigasi Berbasis Web.....	7
C. Konsep Shortest Path.....	7
D. Algoritma Dijkstra.....	8
E. OpenStreetMap dan OpenRouteService.....	8
F. Folium	9
G. Flask	9
BAB III	10
HASIL DAN PEMBAHASAN.....	10
A. Membuat Program.....	10
B. Hasil.....	22
BAB IV	30
PENUTUP.....	30
A. Kesimpulan.....	30
B. Saran.....	30
DAFTAR PUSTAKA	31

DAFTAR GAMBAR

Gambar 1. Import Library	10
Gambar 2. Basis Data Lokasi Kampus	11
Gambar 3. Avoid Polygon	13
Gambar 4. Pemilihan Moda Transportasi	14
Gambar 5. Simulasi dan Deteksi Waktu	15
Gambar 6. Logika Operasional Gerbang Kampus	17
Gambar 7. Implementasi Algoritma Shortest Path (Dijkstra).....	19
Gambar 8. Pembuatan Peta Interaktif	20
Gambar 9. Menampilkan Hasil ke Halaman Web	21
Gambar 10. Tampilan Sistem.....	22
Gambar 11. Pejalan Kaki dari GB 5 Menuju Dekanat Teknik	25
Gambar 12. Sepeda Motor dari GB 5 Menuju Dekanat Teknik	25
Gambar 13. Mobil dari GB 5 Menuju Dekanat Teknik	26
Gambar 14. Sepeda Motor dari GB 5 Menuju Gerbang Keluar Belakang	26
Gambar 15. Sepeda Motor dari GB 5 Menuju Gerbang Keluar Belakang Di Hari Kerja dan Di Luar Jam Operasional Kampus	27
Gambar 16. Sepeda Motor dari GB 5 Menuju Gerbang Keluar Belakang Di Hari Libur.....	27

BAB 1

PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi informasi telah mendorong pemanfaatan sistem navigasi digital dalam berbagai bidang, termasuk lingkungan pendidikan. Sistem navigasi memungkinkan pengguna untuk menemukan lokasi tujuan dengan lebih cepat melalui perhitungan rute perjalanan yang optimal. Salah satu permasalahan yang sering dihadapi pada lingkungan kampus yang memiliki area luas adalah kesulitan dalam menemukan gedung, laboratorium, fakultas, maupun fasilitas pendukung lainnya, terutama bagi mahasiswa baru dan pengunjung kampus.

Universitas Bengkulu merupakan salah satu perguruan tinggi yang memiliki banyak gedung dan fasilitas yang tersebar pada berbagai titik lokasi. Kondisi tersebut menyebabkan pengguna sering mengalami kesulitan dalam menentukan jalur tercepat menuju lokasi tujuan. Selain itu, adanya beberapa gerbang kampus yang memiliki jam operasional tertentu mengakibatkan tidak semua jalur dapat digunakan setiap saat. Oleh karena itu, diperlukan suatu sistem yang mampu memberikan informasi rute yang optimal sesuai dengan kondisi aktual di lingkungan kampus.

Permasalahan pencarian jalur optimal dapat diselesaikan menggunakan konsep Shortest Path Problem, yaitu metode pencarian lintasan dengan jarak atau biaya minimum dari suatu titik asal menuju titik tujuan. Salah satu algoritma yang paling banyak digunakan untuk menyelesaikan permasalahan shortest path adalah algoritma Dijkstra. Algoritma ini bekerja dengan mencari jalur berbiaya minimum pada suatu graf hingga diperoleh rute terpendek yang dapat digunakan pengguna.

Dalam penelitian ini dikembangkan sebuah sistem navigasi kampus berbasis web menggunakan framework Flask yang terintegrasi dengan OpenRouteService dan Folium. Sistem memanfaatkan konsep shortest path dan algoritma Dijkstra untuk menghasilkan rute perjalanan yang optimal berdasarkan lokasi awal dan lokasi tujuan yang dipilih pengguna. Selain itu, sistem juga mempertimbangkan kondisi operasional gerbang kampus sehingga rute yang dihasilkan lebih sesuai dengan kondisi sebenarnya.

Dengan adanya sistem ini diharapkan pengguna dapat memperoleh informasi rute perjalanan yang lebih efektif, efisien, dan akurat dalam lingkungan Universitas Bengkulu sehingga mobilitas di dalam kampus dapat berjalan dengan lebih baik.

B. Rumusan Masalah

1. Bagaimana merancang dan membangun sistem navigasi kampus berbasis web menggunakan konsep Shortest Path pada lingkungan Universitas Bengkulu?
2. Bagaimana menerapkan algoritma Dijkstra untuk menentukan rute terpendek dari lokasi awal menuju lokasi tujuan?
3. Bagaimana mengintegrasikan kondisi operasional gerbang kampus ke dalam proses pencarian rute sehingga menghasilkan jalur yang lebih optimal?
4. Bagaimana menampilkan hasil pencarian rute dalam bentuk peta interaktif yang mudah dipahami oleh pengguna?

C. Tujuan

1. Merancang dan mengembangkan sistem navigasi kampus berbasis web di lingkungan Universitas Bengkulu.
2. Mengimplementasikan konsep Shortest Path menggunakan algoritma Dijkstra untuk menentukan rute perjalanan yang optimal.
3. Mengintegrasikan informasi operasional gerbang kampus dalam proses pencarian jalur.
4. Menyajikan hasil pencarian rute melalui visualisasi peta interaktif sehingga memudahkan pengguna dalam menentukan jalur perjalanan.

D. Manfaat

1. Membantu mahasiswa, dosen, dan pengunjung kampus dalam menemukan lokasi tujuan dengan lebih cepat.
2. Menyediakan informasi rute terpendek yang mempertimbangkan kondisi akses jalan dan operasional gerbang kampus.
3. Menambah wawasan mengenai penerapan algoritma Dijkstra dan konsep Shortest Path dalam sistem navigasi.

BAB II

LANDASAN TEORI

A. Sistem Informasi Geografis

Sistem Informasi Geografis (SIG) atau Geographic Information System (GIS) merupakan sistem yang digunakan untuk mengumpulkan, mengelola, menganalisis, dan menampilkan data yang memiliki informasi geografis atau spasial. GIS memungkinkan pengguna untuk melakukan analisis lokasi serta visualisasi data berbasis peta secara efektif (Taddesse & Tofu, 2025).

Dalam sistem navigasi, GIS berfungsi sebagai media untuk merepresentasikan lokasi, jaringan jalan, dan hubungan antar titik dalam bentuk data spasial sehingga dapat digunakan untuk proses pencarian rute perjalanan.

B. Sistem Navigasi Berbasis Web

Sistem navigasi berbasis web merupakan aplikasi yang menyediakan layanan pencarian lokasi dan rute perjalanan melalui jaringan internet menggunakan teknologi pemetaan digital. Sistem ini memungkinkan pengguna memperoleh informasi lokasi secara real-time melalui browser tanpa memerlukan perangkat lunak tambahan (Paidi et al., 2023).

Penggunaan sistem navigasi berbasis web pada lingkungan kampus dapat membantu pengguna menemukan lokasi tujuan secara lebih cepat dan efisien, terutama pada area kampus yang memiliki banyak gedung dan fasilitas. Kemudian juga mengurangi kemungkinan untuk tersesat atau pengambilan rute yang ternyata lebih jauh.

C. Konsep Shortest Path

Shortest Path atau jalur terpendek merupakan permasalahan pencarian lintasan dengan biaya minimum dari suatu titik asal menuju titik tujuan pada sebuah graf. Biaya yang dimaksud dapat berupa jarak, waktu tempuh, maupun bobot lainnya yang digunakan sebagai parameter pencarian rute

Konsep shortest path menjadi dasar utama dalam berbagai sistem navigasi modern karena mampu memberikan jalur yang lebih efisien dibandingkan jalur lainnya. Dalam sistem navigasi kampus, shortest path digunakan untuk menentukan rute terbaik dari lokasi awal menuju lokasi tujuan berdasarkan jaringan jalan yang tersedia di lingkungan tersebut.

D. Algoritma Dijkstra

Algoritma Dijkstra merupakan salah satu algoritma pencarian jalur terpendek yang diperkenalkan oleh Edsger W. Dijkstra. Algoritma ini bekerja dengan mencari lintasan berbiaya minimum dari titik sumber ke seluruh titik lain dalam suatu graf berbobot non-negatif

Proses kerja algoritma Dijkstra dimulai dengan memberikan nilai jarak awal nol pada titik sumber dan nilai tak hingga pada titik lainnya. Selanjutnya algoritma akan memilih simpul dengan jarak terkecil yang belum dikunjungi dan memperbarui jarak simpul tetangganya hingga seluruh simpul selesai diproses.

Keunggulan algoritma Dijkstra terletak pada kemampuannya menghasilkan jalur optimal dengan tingkat akurasi yang tinggi serta efisiensi yang baik untuk jaringan jalan berukuran sedang hingga besar. Oleh karena itu, algoritma ini banyak digunakan dalam aplikasi navigasi, sistem transportasi, dan Geographic Information System (GIS). Adapun kekurangan dari algoritma ini ialah Algoritma ini menyebar ke segala arah secara bersamaan, hal ini memakan memori komputasi yang besar jika petanya berukuran raksasa.

Pada penelitian ini, konsep algoritma Dijkstra digunakan sebagai dasar pencarian rute terpendek yang diterapkan melalui layanan OpenRouteService sehingga sistem mampu menghasilkan jalur optimal antara lokasi awal dan lokasi tujuan.

E. OpenStreetMap dan OpenRouteService

OpenStreetMap (OSM) merupakan basis data peta digital terbuka yang menyediakan informasi jalan, bangunan, dan berbagai objek geografis lainnya yang dapat digunakan secara bebas dalam pengembangan aplikasi GIS.

OpenRouteService (ORS) merupakan layanan routing berbasis OpenStreetMap yang menyediakan berbagai fitur navigasi seperti pencarian rute, estimasi waktu tempuh, dan alternatif jalur perjalanan. Dalam implementasinya, OpenRouteService memanfaatkan konsep shortest path dan algoritma routing untuk menghasilkan rute yang optimal (Giraud, 2022).

Pada sistem yang dikembangkan, OpenRouteService digunakan sebagai mesin utama untuk menghitung rute perjalanan berdasarkan lokasi awal dan lokasi tujuan yang dipilih pengguna.

F. Folium

Folium merupakan pustaka Python yang digunakan untuk membangun peta interaktif berbasis Leaflet.js. Folium memungkinkan pengembang menampilkan marker lokasi, jalur perjalanan, polygon, dan berbagai objek geografis lainnya secara visual pada halaman web (Widyantoro & Prayogo, 2023).

Dalam penelitian ini, Folium digunakan untuk menampilkan lokasi gedung, gerbang kampus, serta visualisasi rute hasil perhitungan shortest path sehingga pengguna dapat memahami jalur perjalanan secara lebih mudah.

G. Flask

Flask merupakan framework web berbasis Python yang bersifat ringan dan fleksibel. Framework ini banyak digunakan untuk pengembangan aplikasi web karena mudah diintegrasikan dengan berbagai pustaka dan layanan eksternal.

Dalam sistem navigasi kampus yang dikembangkan, Flask berfungsi sebagai backend yang mengelola input pengguna, memproses permintaan pencarian rute, menjalankan perhitungan shortest path melalui OpenRouteService, serta menampilkan hasil visualisasi peta kepada pengguna.

BAB III

HASIL DAN PEMBAHASAN

A. Membuat Program

1. Import Library

PrintScreen Source Code:

```
from flask import Flask, render_template, request
import openrouteservice
import folium
from datetime import datetime
```

GIA024055

Gambar 1. Import Library

Source Code:

```
from flask import Flask,
render_
template,
request
import openrouteservice
import folium
from datetime import datetime
```

Penjelasan Source Code:

Kode tersebut berfungsi mengimpor library yang diperlukan dalam pengembangan aplikasi web berbasis Python. `from flask import Flask`, Library Flask digunakan untuk membangun aplikasi web, menampilkan halaman HTML, dan memproses input pengguna.

Library OpenRouteService `import openrouteservice` digunakan untuk menghitung rute, jarak, dan informasi perjalanan berdasarkan data geografis. Library Folium berfungsi menampilkan peta interaktif beserta lokasi dan rute perjalanan secara visual.

Sementara itu, `from datetime import datetime` digunakan untuk mengelola data tanggal dan waktu yang diperlukan dalam proses perhitungan maupun pencatatan aktivitas aplikasi. Kombinasi library tersebut memungkinkan sistem menyediakan layanan pemetaan dan navigasi yang dapat diakses melalui antarmuka web secara interaktif.

2. Basis Data Lokasi Kampus

PrintScreen Source Code:

```
locations = {
  'Asrama PGSD' : [102.27160069043919, -3.7617418863983096],
  'Danau Unib' : [102.27306446726114, -3.758445851629117],
  'Dekanat FEB' : [102.26862389169713, -3.7617198090691164],
  'Dekanat FISIP' : [102.27417328919393, -3.7590310709254973],
  'Dekanat FKIP' : [102.27504444750583, -3.75753414150989],
  'Dekanat FMIPA' : [102.2747136045303, -3.756028855847586],
  'Dekanat Hukum' : [102.26844172291675, -3.760583199392584 ],
  'Dekanat Pertanian' : [102.26921964529129, -3.759336210212976 ],
  'Dekanat Teknik' : [102.27670099113969, -3.7584642603667104 ],
  'Fakultas Kedokteran' : [ 102.27803206102215, -3.7551337561874982],
  'GB 1' : [102.27372095056845, -3.7568032921655625],
  'GB 2' : [102.274037554275, -3.7578575751002457 ],
  'GB 3 & 4' : [102.27664495427499, -3.7560850630710587],
  'GB 5' : [102.27650213893024, -3.7553463918453187],
  'GLT' : [102.27191958168342, -3.75809920273443 ],
  'GSG' : [ 102.27655797563433, -3.757536160753844 ],
  'Gedung C' : [102.26791776641025, -3.7590706965117002],
  'Gedung FKIP' : [102.27746551161644, -3.756364599659421],
  'Gedung Fisika' : [102.27372386940291, -3.7562055013818023],
  'Gedung J' : [102.2697707104651, -3.76030119474263],
  'Gedung K' : [102.26990813916206, -3.761142906184093],
  'Gerbang Keluar Belakang' : [102.27618970824084, -3.7592446156990227],
  'Gerbang Keluar Depan' : [102.26666968496903, -3.758831903679123],
  'Gerbang Masuk Belakang' : [102.27521569504947, -3.759614988399855],
  'Gerbang Masuk Depan' : [102.26779832443637, -3.7597051616151904],
  'Gerbang Masuk Rektorat' : [102.27268704582688, -3.760605072804902],
  'Jurusan Ekonomi Pembangunan' : [102.26894613829127, -3.7617576387695197],
  'Klinik Pratama Unib' : [102.2717675810424, -3.7614596037430554],
  'LAB Agronomi' : [102.27271362771398, -3.7570165757307543 ],
  'LAB Hukum' : [102.26867339480454, -3.7602660084096446 ],
  'LAB Ilmu Tanah' : [102.27012662561205, -3.7592326497010897 ],
  'LAB Teknik' : [102.27690975882886, -3.758891053967651 ],
  'LAB Terpadu Teknik' : [102.27735016612473, -3.7585892834199925],
  'LPTIK' : [102.27501541748192, -3.7585034389347047],
  'Magister Ilmu Ekonomi' : [102.2686381084575, -3.7624574876125974],
}
```

G1A024055

Gambar 2. Basis Data Lokasi Kampus

Source Code:

```
locations = {
  'Asrama PGSD' : [102.27160069043919, -3.7617418863983096],
  'Danau Unib' : [102.27306446726114, -3.758445851629117],
  'Dekanat FEB' : [102.26862389169713, -3.7617198090691164],
  'Dekanat FISIP' : [102.27417328919393, -3.7590310709254973],
  'Dekanat FKIP' : [102.27504444750583, -3.75753414150989],
  'Dekanat FMIPA' : [102.2747136045303, -3.756028855847586],
  'Dekanat Hukum' : [102.26844172291675, -3.760583199392584 ],
  'Dekanat Pertanian' : [102.26921964529129, -3.759336210212976 ],
  'Dekanat Teknik' : [102.27670099113969, -3.7584642603667104 ],
```

'Fakultas Kedokteran' : [102.27803206102215, -3.7551337561874982],
 'GB 1' : [102.27372095056845, -3.7568032921655625],
 'GB 2' : [102.274037554275, -3.7578575751002457],
 'GB 3 & 4' : [102.27664495427499, -3.7560850630710587],
 'GB 5' : [102.27650213893024, -3.7553463918453187],
 'GLT' : [102.27191958168342, -3.75809920273443],
 'GSG' : [102.27655797563433, -3.757536160753844],
 'Gedung C' : [102.26791776641025, -3.7590706965117002],
 'Gedung FKIP' : [102.27746551161644, -3.756364599659421],
 'Gedung Fisika' : [102.27372386940291, -3.7562055013818023],
 'Gedung J' : [102.2697707104651, -3.76030119474263],
 'Gedung K' : [102.26990813916206, -3.761142906184093],
 'Gerbang Keluar Belakang' : [102.27618970824084, -
 3.7592446156990227],
 'Gerbang Keluar Depan' : [102.26666968496903, -3.758831903679123],
 'Gerbang Masuk Belakang' : [102.27521569504947, -
 3.759614988399855],
 'Gerbang Masuk Depan' : [102.26779832443637, -3.7597051616151904],
 'Gerbang Masuk Rektorat' : [102.27268704582688, -3.760605072804902],
 'Jurusan Ekonomi Pembangunan' : [102.26894613829127, -
 3.7617576387695197],
 'Klinik Pratama Unib' : [102.2717675810424, -3.7614596037430554],
 'LAB Agronomi' : [102.27271362771398, -3.7570165757307543],
 'LAB Hukum' : [102.26867339480454, -3.7602660084096446],
 'LAB Ilmu Tanah' : [102.27012662561205, -3.7592326497010897],
 'LAB Teknik' : [102.27690975882886, -3.758891053967651],
 'LAB Terpadu Teknik' : [102.27735016612473, -3.7585892834199925],
 'LPTIK' : [102.27501541748192, -3.7585034389347047],
 'Magister Ilmu Ekonomi' : [102.2686381084575, -3.7624574876125974],
 'Masjid Baitul Hikmah' : [102.27600666694858, -3.758945132312725],
 'Masjid Darul Ulum' : [102.2675868394383, -3.757278224804399],
 'Mushola Shelter' : [102.27361705847183, -3.7576982064708235],

```
'Perpustakaan' : [102.27485462111163, -3.756806076798016],
'Rektorat' : [102.27231460986346, -3.7590495172423495],
'Ruang Baca Pertanian' : [102.27283662942735, -3.7571162998364276],
'S2 Matematika' : [102.27543890078353, -3.7580570001381477],
'Sekretariat BEM FMIPA' : [102.27496036775979, -3.75578529907203],
'Sekretariat Teknik' : [102.27733334828312, -3.7581980275566127 ],
'Sekretariat UKM' : [102.2757012915378, -3.756636058655066],
'Stadion Unib' : [102.27817155070424, -3.7576442412116946],
'UPT Bing' : [102.27036307553568, -3.7607740664372096]}
lokasi_berurutan = sorted(locations.keys())
```

Penjelasan Source Code:

Kode tersebut berisi struktur data dictionary bernama **locations** yang digunakan untuk menyimpan daftar titik lokasi dalam bentuk koordinat geografis (longitude dan latitude). Setiap lokasi direpresentasikan oleh pasangan *key-value*, di mana *key* berisi nama gedung atau titik penting di lingkungan Universitas Bengkulu, seperti Asrama PGSD, Dekanat, Gedung Kuliah, Gedung FKIP, Gedung Rektorat, Perpustakaan, hingga Laboratorium. Sementara itu, *value* berisi koordinat lokasi yang digunakan sebagai acuan dalam proses pemetaan.

Data koordinat ini menjadi dasar bagi sistem untuk menampilkan marker pada peta, menentukan posisi suatu lokasi, serta menghitung rute dan jarak perjalanan antar titik menggunakan layanan pemetaan yang terintegrasi dalam aplikasi. Dengan adanya struktur data ini, seluruh lokasi dapat dikelola secara terpusat sehingga memudahkan proses pencarian dan visualisasi lokasi pada sistem navigasi kampus.

3. Fungsi Pembentukan Area Larangan (Avoid Polygon)

PrintScreen Source Code:

```
def buat_polygon_hindar(coord, offset=0.00015):
    lon, lat = coord
    return [[lon - offset, lat - offset], [lon + offset, lat - offset],
            [lon + offset, lat + offset], [lon - offset, lat + offset],
            [lon - offset, lat - offset]]
```

G1A024055

Gambar 3. Avoid Polygon

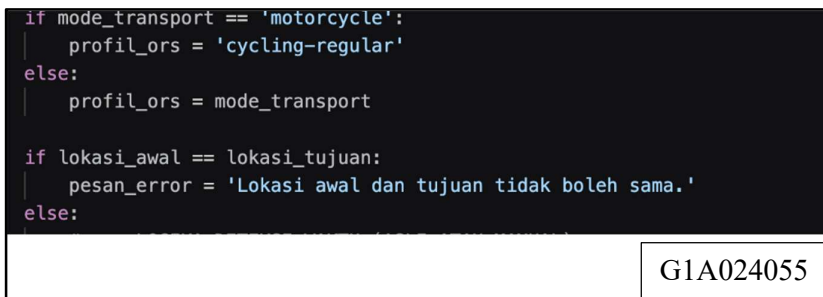
Source Code:

```
def buat_polygon_hindar(coord, offset=0.00015):  
    lon, lat = coord  
    return [[[lon - offset, lat - offset], [lon + offset, lat - offset],  
            [lon + offset, lat + offset], [lon - offset, lat + offset],  
            [lon - offset, lat - offset]]]
```

Penjelasan Source Code:

Fungsi `buat_polygon_hindar()` digunakan untuk membuat area berbentuk poligon persegi di sekitar suatu titik koordinat tertentu. Fungsi menerima parameter `coord` yang berisi koordinat lokasi (longitude dan latitude) serta parameter `offset` yang menentukan ukuran area di sekitar titik tersebut. Nilai longitude dan latitude kemudian dipisahkan ke dalam variabel `lon` dan `lat`.

Selanjutnya, fungsi membentuk lima titik koordinat yang menyusun sebuah persegi mengelilingi titik pusat dengan jarak sesuai nilai `offset`. Titik pertama diulang kembali pada akhir daftar untuk menutup bentuk poligon. Hasil dari fungsi ini berupa koordinat poligon yang dapat digunakan sebagai area yang harus dihindari (*avoid polygon*) dalam proses pencarian rute. Dengan demikian, sistem dapat mengarahkan pengguna agar tidak melewati lokasi atau zona tertentu yang telah ditentukan.

4. Pemilihan Mode Transportasi**PrintScreen Source Code:**

```
if mode_transport == 'motorcycle':  
    profil_ors = 'cycling-regular'  
else:  
    profil_ors = mode_transport  
  
if lokasi_awal == lokasi_tujuan:  
    pesan_error = 'Lokasi awal dan tujuan tidak boleh sama.'  
else:
```

G1A024055

Gambar 4. Pemilihan Moda Transportasi

Source Code:

```
if mode_transport == 'motorcycle':  
    profil_ors = 'cycling-regular'
```

```

else:
    profil_ors = mode_transport
    if lokasi_awal == lokasi_tujuan:
        pesan_error = 'Lokasi awal dan tujuan tidak boleh sama.'
    else:

```

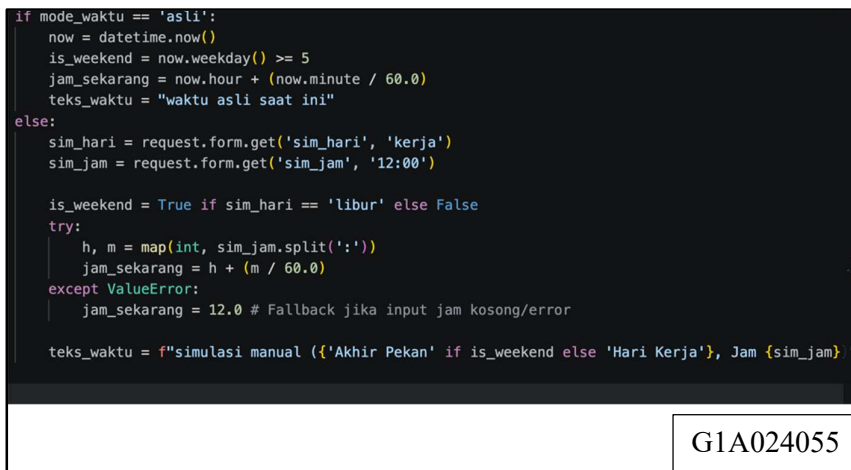
Penjelasan Source Code:

Potongan kode tersebut digunakan untuk menentukan profil transportasi yang akan digunakan dalam perhitungan rute serta melakukan validasi lokasi yang dipilih pengguna. Jika pengguna memilih moda transportasi motorcycle, sistem akan mengubahnya menjadi **cycling-regular**, yaitu profil yang tersedia pada layanan OpenRouteService untuk merepresentasikan perjalanan kendaraan roda dua.

Jika pengguna memilih moda transportasi lain, sistem akan langsung menggunakan nilai yang dipilih sebagai profil rute. Selanjutnya, sistem memeriksa apakah lokasi awal dan lokasi tujuan yang dipilih pengguna sama. Apabila keduanya identik, sistem akan menampilkan pesan kesalahan berupa *"Lokasi awal dan tujuan tidak boleh sama"* untuk mencegah proses pencarian rute yang tidak valid. Jika lokasi berbeda, maka proses perhitungan dan pencarian rute dapat dilanjutkan ke tahap berikutnya.

5. Simulasi dan Deteksi Waktu

PrintScreen Source Code:



```

if mode_waktu == 'asli':
    now = datetime.now()
    is_weekend = now.weekday() >= 5
    jam_sekarang = now.hour + (now.minute / 60.0)
    teks_waktu = "waktu asli saat ini"
else:
    sim_hari = request.form.get('sim_hari', 'kerja')
    sim_jam = request.form.get('sim_jam', '12:00')

    is_weekend = True if sim_hari == 'libur' else False
    try:
        h, m = map(int, sim_jam.split(':'))
        jam_sekarang = h + (m / 60.0)
    except ValueError:
        jam_sekarang = 12.0 # Fallback jika input jam kosong/error

    teks_waktu = f"simulasi manual ({'Akhir Pekan' if is_weekend else 'Hari Kerja'}, Jam {sim_jam})"

```

G1A024055

Gambar 5. Simulasi dan Deteksi Waktu

Source Code:

```
if mode_waktu == 'asli':
    now = datetime.now()
    is_weekend = now.weekday() >= 5
    jam_sekarang = now.hour + (now.minute / 60.0)
    teks_waktu = "waktu asli saat ini"
else:
    sim_hari = request.form.get('sim_hari', 'kerja')
    sim_jam = request.form.get('sim_jam', '12:00')
    is_weekend = True if sim_hari == 'libur' else False
    try:
        h, m = map(int, sim_jam.split(':'))
        jam_sekarang = h + (m / 60.0)
    except ValueError:
        jam_sekarang = 12.0 # Fallback jika input jam kosong/error
    teks_waktu = f'simulasi manual ({'Akhir Pekan' if is_weekend else 'Hari Kerja'}, Jam {sim_jam})"
```

Penjelasan Source Code:

Potongan kode tersebut digunakan untuk menentukan waktu yang akan dijadikan acuan dalam proses perhitungan rute. Jika pengguna memilih mode asli, sistem akan mengambil tanggal dan waktu saat ini menggunakan fungsi `datetime.now()`. Selanjutnya, sistem menentukan apakah hari tersebut termasuk akhir pekan dengan memeriksa nilai `weekday()`, serta menghitung waktu dalam format desimal berdasarkan jam dan menit saat ini. Informasi ini kemudian disimpan sebagai keterangan bahwa perhitungan menggunakan waktu nyata (*real-time*).

Sebaliknya, jika pengguna memilih mode simulasi, sistem akan mengambil data hari dan jam yang dimasukkan melalui formulir. Nilai hari digunakan untuk menentukan apakah simulasi dilakukan pada hari kerja atau akhir pekan, sedangkan nilai jam dikonversi menjadi format desimal agar dapat digunakan dalam proses perhitungan. Untuk mengantisipasi kesalahan atau kekosongan input, program menggunakan blok `try-except`

sehingga jika terjadi error, sistem akan menggunakan nilai default pukul 12.00. Selanjutnya, informasi simulasi disimpan dalam variabel **teks_waktu** yang berisi jenis hari dan jam yang digunakan. Dengan mekanisme ini, sistem dapat melakukan perhitungan rute baik berdasarkan kondisi waktu sebenarnya maupun berdasarkan skenario waktu yang ditentukan pengguna.

6. Logika Operasional Gerbang Kampus

PrintScreen Source Code:



```
gerbang_tutup = []
is_jam_operasional = (not is_weekend) and (6.0 <= jam_sekarang < 18.0)

if not is_jam_operasional:
    gerbang_tutup.extend(['Gerbang Masuk Depan', 'Gerbang Keluar Depan', 'Gerbang Keluar Belakang',
    pesan_info = f'Berdasarkan {teks_waktu}, kampus di luar jam operasional. Rute dihitung via Gerl
else:
    pesan_info = f'Berdasarkan {teks_waktu}, semua gerbang beroperasi normal. '

if lokasi_awal in gerbang_tutup or lokasi_tujuan in gerbang_tutup:
    pesan_error = f'Pencarian dibatalkan: Lokasi yang dipilih sedang TUTUP pada {teks_waktu}.'
else:
    avoid_polygons = [buat_polygon_hindar(locations[g]) for g in gerbang_tutup if g in locations]
    route_coords = [locations[lokasi_awal], locations[lokasi_tujuan]]
```

G1A024055

Gambar 6. Logika Operasional Gerbang Kampus

Source Code:

```
gerbang_tutup = []
is_jam_operasional = (not is_weekend) and (6.0 <= jam_sekarang < 18.0)
if not is_jam_operasional:
    gerbang_tutup.extend(['Gerbang Masuk Depan', 'Gerbang Keluar Depan',
    'Gerbang Keluar Belakang', 'Gerbang Masuk Rektorat'])
    pesan_info = f'Berdasarkan {teks_waktu}, kampus di luar jam operasional.
    Rute dihitung via Gerbang Masuk Belakang. '
else:
    pesan_info = f'Berdasarkan {teks_waktu}, semua gerbang beroperasi
    normal. '
if lokasi_awal in gerbang_tutup or lokasi_tujuan in gerbang_tutup:
    pesan_error = f'Pencarian dibatalkan: Lokasi yang dipilih sedang TUTUP
    pada {teks_waktu}.'
else:
    avoid_polygons = [buat_polygon_hindar(locations[g]) for g in
    gerbang_tutup if g in locations]
```

```
route_coords = [locations[lokasi_awal], locations[lokasi_tujuan]]
```

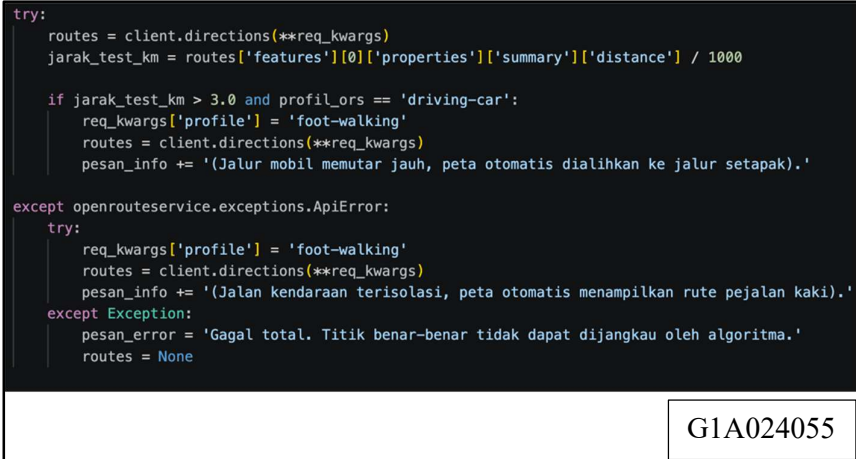
Penjelasan Source Code:

Potongan kode tersebut digunakan untuk mengatur akses gerbang kampus berdasarkan waktu operasional yang berlaku. Variabel `gerbang_tutup` dibuat untuk menyimpan daftar gerbang yang tidak dapat digunakan. Sistem kemudian memeriksa apakah waktu saat ini atau waktu simulasi berada pada jam operasional kampus, yaitu hari kerja antara pukul 06.00 hingga 18.00. Jika berada di luar jam operasional, beberapa gerbang seperti Gerbang Masuk Depan, Gerbang Keluar Depan, Gerbang Keluar Belakang, dan Gerbang Masuk Rektorat akan dimasukkan ke dalam daftar gerbang tertutup. Pada kondisi ini, sistem memberikan informasi bahwa rute akan diarahkan melalui Gerbang Masuk Belakang. Sebaliknya, jika masih dalam jam operasional, sistem menginformasikan bahwa seluruh gerbang dapat digunakan secara normal.

Selanjutnya, sistem melakukan validasi terhadap lokasi awal dan tujuan yang dipilih pengguna. Jika salah satu lokasi berada pada daftar gerbang yang sedang ditutup, proses pencarian rute dibatalkan dan sistem menampilkan pesan kesalahan. Namun, jika lokasi yang dipilih masih dapat diakses, sistem akan membuat area larangan (*avoid polygon*) di sekitar setiap gerbang yang ditutup menggunakan fungsi `buat_polygon_hindar()`. Area ini digunakan agar algoritma pencarian rute tidak melewati gerbang yang sedang tidak beroperasi. Setelah itu, koordinat lokasi awal dan tujuan disimpan ke dalam variabel `route_coords` sebagai data utama yang akan digunakan dalam proses perhitungan rute perjalanan.

7. Implementasi Algoritma Shortest Path (Dijkstra)

PrintScreen Source Code:



```
try:
    routes = client.directions(**req_kwargs)
    jarak_test_km = routes['features'][0]['properties']['summary']['distance'] / 1000

    if jarak_test_km > 3.0 and profil_ors == 'driving-car':
        req_kwargs['profile'] = 'foot-walking'
        routes = client.directions(**req_kwargs)
        pesan_info += '(Jalur mobil memutar jauh, peta otomatis dialihkan ke jalur setapak).'
```

G1A024055

Gambar 7. Implementasi Algoritma Shortest Path (Dijkstra)

Source Code:

```
try: routes = client.directions(**req_kwargs)
jarak_test_km = routes['features'][0]['properties']['summary']['distance'] /
1000
if jarak_test_km > 3.0 and profil_ors == 'driving-car':
req_kwargs['profile'] = 'foot-walking'
routes = client.directions(**req_kwargs)
pesan_info += '(Jalur mobil memutar jauh, peta otomatis dialihkan ke jalur
setapak).'
```

```
except openrouteservice.exceptions.ApiError:
try:
req_kwargs['profile'] = 'foot-walking'
routes = client.directions(**req_kwargs)
pesan_info += '(Jalan kendaraan terisolasi, peta otomatis menampilkan rute
pejalan kaki).'
```

```
except Exception:
pesan_error = 'Gagal total. Titik benar-benar tidak dapat dijangkau oleh
algoritma.'
```

```
routes = None
```

Penjelasan Source Code:

Potongan kode tersebut berfungsi untuk menghitung rute perjalanan menggunakan layanan OpenRouteService sekaligus menangani kondisi ketika rute kendaraan tidak tersedia atau kurang efisien. Pada tahap awal, sistem mengirim permintaan pencarian rute melalui fungsi `client.directions()` dan menghitung total jarak perjalanan yang diperoleh. Jika jarak rute yang dihasilkan melebihi 3 kilometer saat menggunakan profil kendaraan (`driving-car`), sistem menganggap jalur tersebut terlalu memutar sehingga secara otomatis mengubah metode pencarian menjadi jalur pejalan kaki (`foot-walking`). Setelah rute baru berhasil diperoleh, sistem menambahkan informasi bahwa peta telah dialihkan ke jalur setapak agar perjalanan menjadi lebih efisien.

Selain itu, kode menggunakan blok `try-except` untuk menangani kemungkinan kegagalan dalam proses pencarian rute. Jika OpenRouteService mengembalikan kesalahan (`ApiError`), sistem akan mencoba kembali mencari rute menggunakan profil pejalan kaki sebagai alternatif. Apabila pencarian alternatif berhasil, sistem memberikan informasi bahwa lokasi hanya dapat dijangkau melalui jalur pejalan kaki. Namun, jika seluruh percobaan tetap gagal, sistem akan menampilkan pesan kesalahan yang menyatakan bahwa titik tujuan tidak dapat dijangkau oleh algoritma pemetaan dan variabel `routes` akan diatur menjadi `None`. Mekanisme ini membuat aplikasi lebih adaptif dan mampu memberikan solusi otomatis ketika rute kendaraan tidak tersedia.

8. Pembuatan Peta Interaktif

PrintScreen Source Code:



Gambar 8. Pembuatan Peta Interaktif

Source Code:

```
if not pesan_error and routes:
    m = folium.Map(location=[-3.758, 102.272], zoom_start=16,
    tiles='CartoDB positron')
```

Penjelasan Source Code:

Potongan kode tersebut digunakan untuk memastikan bahwa proses visualisasi peta hanya dilakukan apabila tidak terdapat kesalahan (`pesan_error`) dan data rute berhasil diperoleh (`routes`). Jika kedua syarat tersebut terpenuhi, sistem akan membuat objek peta baru menggunakan library Folium melalui fungsi `folium.Map()`.

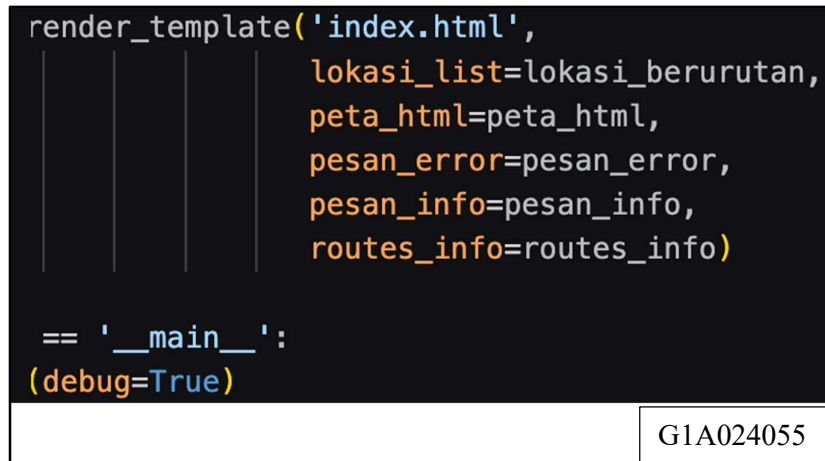
Peta diinisialisasi dengan titik pusat pada koordinat `[-3.758, 102.272]`, yang merupakan area kampus Universitas Bengkulu, serta menggunakan tingkat perbesaran (*zoom level*) sebesar 16 agar detail lokasi dapat terlihat dengan jelas. Selain itu, peta menggunakan jenis tampilan (*tiles*) CartoDB Positron, yaitu peta dengan desain sederhana dan bersih yang memudahkan pengguna dalam melihat lokasi, marker, maupun rute perjalanan yang akan ditampilkan pada tahap selanjutnya. Dengan demikian, peta berfungsi sebagai media visual untuk menyajikan hasil perhitungan rute secara interaktif kepada pengguna.

9. Menampilkan Hasil ke Halaman Web

PrintScreen Source Code:

```
render_template('index.html',
                lokasi_list=lokasi_berurutan,
                peta_html=peta_html,
                pesan_error=pesan_error,
                pesan_info=pesan_info,
                routes_info=routes_info)

== '__main__':
(debug=True)
```



Gambar 9. Menampilkan Hasil ke Halaman Web

Source Code:

```
return render_template('index.html',
lokasi_list=lokasi_berurutan,
peta_html=peta_html,
pesan_error=pesan_error,
```

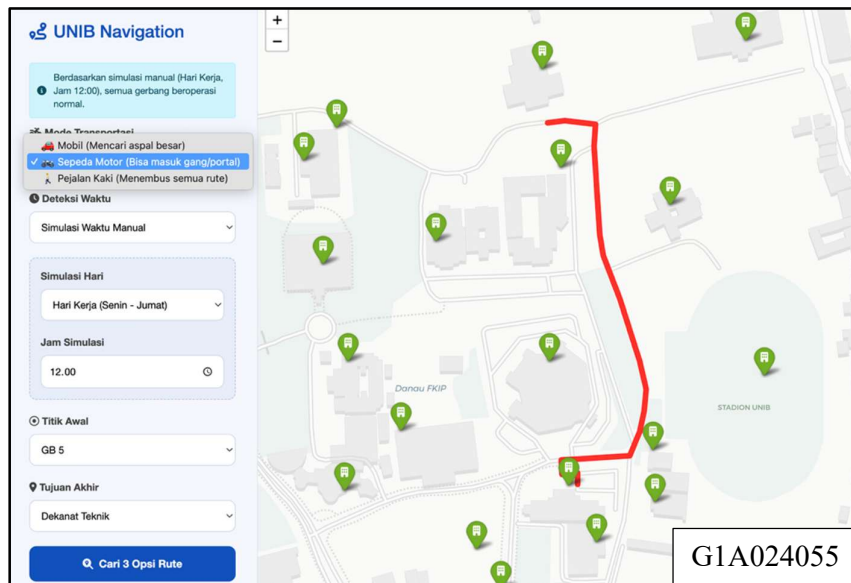
```
pesan_info=pesan_info,  
routes_info=routes_info)
```

Penjelasan Source Code:

Potongan kode tersebut digunakan untuk mengirim data dari aplikasi Flask ke halaman antarmuka pengguna (`index.html`) menggunakan fungsi `render_template()`. Fungsi ini menampilkan halaman web sekaligus meneruskan berbagai data yang telah diproses oleh sistem. Variabel `lokasi_list` berisi daftar lokasi yang akan ditampilkan pada pilihan pengguna, `peta_html` berisi hasil peta interaktif yang telah dibuat.

`pesan_error` digunakan untuk menampilkan informasi kesalahan apabila terjadi masalah dalam proses pencarian rute, dan `pesan_info` digunakan untuk memberikan informasi tambahan terkait kondisi rute atau status gerbang kampus. Selain itu, variabel `routes_info` berisi detail hasil perhitungan rute, seperti jarak atau informasi perjalanan lainnya. Dengan mekanisme ini, seluruh hasil pemrosesan dapat ditampilkan secara dinamis pada halaman web sehingga pengguna dapat melihat informasi rute, peta, dan pesan sistem dalam satu tampilan yang terintegrasi.

B. Hasil



Gambar 10. Tampilan Sistem

Hasil dari program yang dikembangkan adalah sebuah sistem navigasi kampus berbasis web yang dirancang untuk membantu pengguna

menemukan rute terpendek menuju berbagai lokasi di lingkungan Universitas Bengkulu. Sistem ini dibangun menggunakan framework Flask sebagai backend, OpenRouteService sebagai layanan perhitungan rute, Folium sebagai media visualisasi peta interaktif, serta OpenStreetMap sebagai sumber data geografis. Struktur program terdiri dari file utama Python yang berfungsi sebagai pusat pengolahan data dan logika sistem, folder templates yang berisi halaman antarmuka pengguna, serta berbagai library pendukung yang digunakan dalam proses pengembangan aplikasi. Seluruh komponen tersebut bekerja secara terintegrasi sehingga sistem mampu menerima input pengguna, memproses pencarian jalur, dan menampilkan hasil navigasi dalam bentuk peta digital.

Program dijalankan melalui file utama Python dan akan mengaktifkan server Flask yang dapat diakses melalui browser. Ketika sistem berhasil dijalankan, pengguna akan disajikan halaman utama yang berisi berbagai komponen navigasi, seperti pemilihan moda transportasi, pengaturan simulasi waktu, pemilihan titik awal, serta pemilihan lokasi tujuan. Antarmuka dirancang agar mudah digunakan sehingga pengguna dapat melakukan pencarian rute tanpa memerlukan pengetahuan teknis mengenai sistem yang digunakan.

Pada bagian moda transportasi, pengguna dapat memilih jenis kendaraan yang akan digunakan, yaitu mobil, sepeda motor, atau pejalan kaki. Setiap pilihan transportasi akan memengaruhi proses pencarian jalur karena sistem menggunakan profil perjalanan yang berbeda sesuai karakteristik masing-masing moda transportasi. Selain itu, sistem juga menyediakan fitur simulasi waktu yang memungkinkan pengguna menentukan hari dan jam tertentu. Fitur ini digunakan untuk mensimulasikan kondisi operasional kampus sehingga sistem dapat menyesuaikan jalur yang tersedia berdasarkan waktu yang dipilih.

Setelah pengguna menentukan lokasi awal dan lokasi tujuan, sistem akan mengambil koordinat kedua lokasi tersebut dari basis data lokasi kampus yang telah disediakan. Selanjutnya sistem akan memproses data menggunakan layanan OpenRouteService yang memanfaatkan konsep

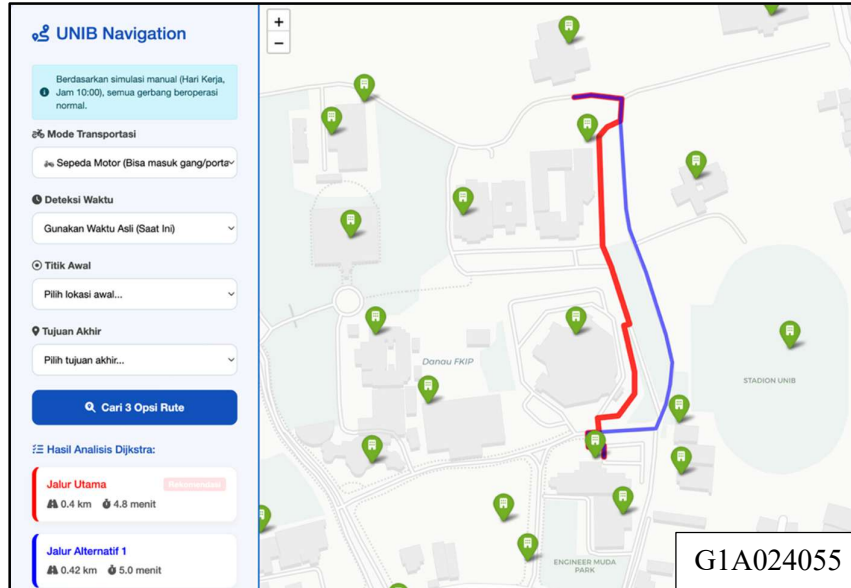
Shortest Path untuk menentukan jalur paling optimal. Dalam proses ini, jaringan jalan direpresentasikan sebagai graf yang terdiri atas simpul dan ruas jalan. Prinsip kerja yang digunakan mengacu pada algoritma Dijkstra, yaitu algoritma pencarian lintasan terpendek yang menentukan jalur dengan biaya minimum dari titik awal menuju titik tujuan. Melalui mekanisme tersebut, sistem mampu menghasilkan rute yang efisien berdasarkan kondisi jalan yang tersedia.

Selain pencarian jalur utama, sistem juga menyediakan beberapa alternatif rute yang dapat digunakan apabila pengguna ingin memilih jalur lain. Setiap rute yang dihasilkan akan dihitung berdasarkan jarak tempuh dan estimasi waktu perjalanan. Sistem kemudian melakukan penyaringan terhadap jalur alternatif yang dianggap terlalu jauh sehingga hanya rute yang relevan dan masuk akal yang ditampilkan kepada pengguna. Proses ini bertujuan untuk meningkatkan kualitas rekomendasi jalur yang diberikan oleh sistem.

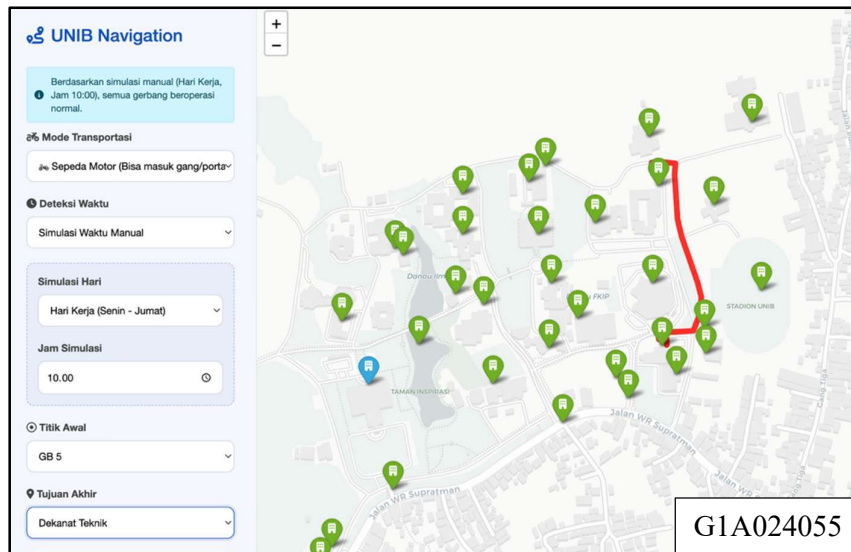
Hasil perhitungan selanjutnya divisualisasikan menggunakan library Folium dalam bentuk peta interaktif. Pada peta tersebut ditampilkan lokasi-lokasi penting di lingkungan Universitas Bengkulu beserta jalur perjalanan yang telah dihitung oleh sistem. Informasi tambahan seperti kondisi operasional gerbang kampus, lokasi awal, lokasi tujuan, jarak tempuh, dan estimasi waktu perjalanan juga ditampilkan untuk membantu pengguna memahami hasil navigasi yang diberikan.

Secara keseluruhan, program ini berhasil mengimplementasikan sistem navigasi kampus berbasis web dengan memanfaatkan konsep Shortest Path dan algoritma Dijkstra dalam proses pencarian rute. Integrasi antara Flask, OpenRouteService, OpenStreetMap, dan Folium memungkinkan sistem menghasilkan jalur yang optimal serta menyajikan informasi navigasi secara interaktif dan mudah dipahami. Dengan adanya sistem ini, pengguna dapat memperoleh bantuan dalam menentukan rute perjalanan yang efisien menuju berbagai lokasi di lingkungan Universitas Bengkulu sehingga proses mobilitas di dalam kampus menjadi lebih mudah dan efektif.

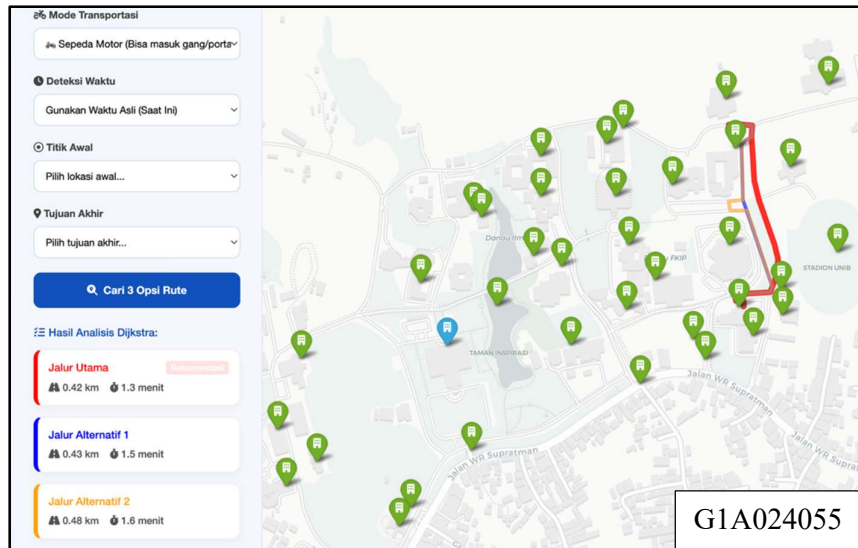
C. Uji Coba



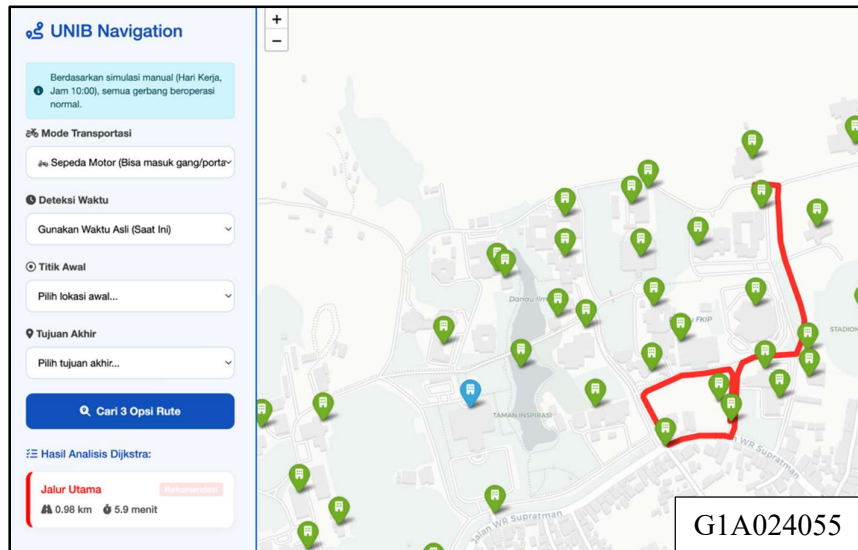
Gambar 11. Pejalan Kaki dari GB 5 Menuju Dekanat Teknik



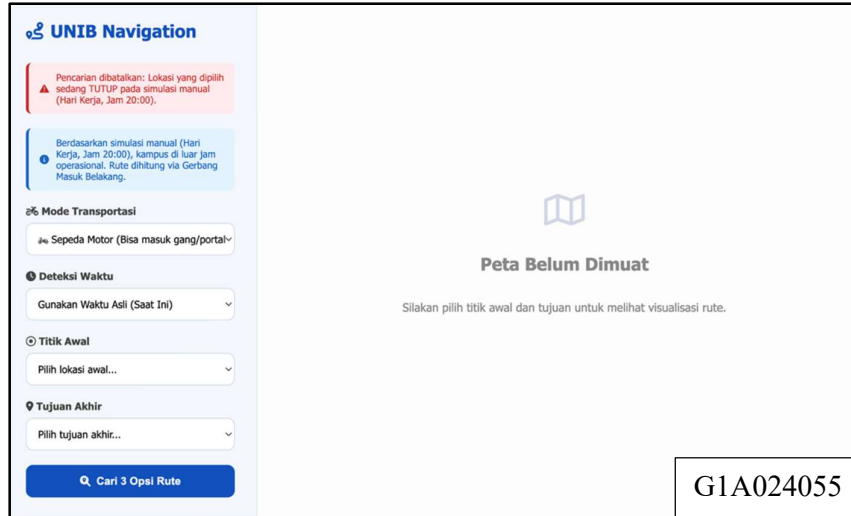
Gambar 12. Sepeda Motor dari GB 5 Menuju Dekanat Teknik



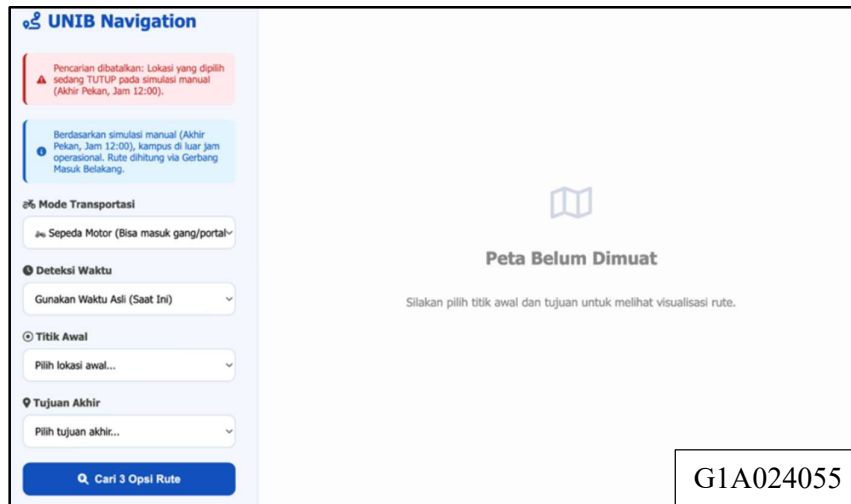
Gambar 13. Mobil dari GB 5 Menuju Dekanat Teknik



Gambar 14. Sepeda Motor dari GB 5 Menuju Gerbang Keluar Belakang



Gambar 15. Sepeda Motor dari GB 5 Menuju Gerbang Keluar Belakang Di Hari Kerja dan Di Luar Jam Operasional Kampus



Gambar 16. Sepeda Motor dari GB 5 Menuju Gerbang Keluar Belakang Di Hari Libur

Berdasarkan Gambar 11 hingga Gambar 16, hasil pengujian sistem menunjukkan bahwa aplikasi berhasil melakukan proses pencarian dan visualisasi rute di lingkungan Kampus Universitas Bengkulu dengan berbagai kondisi penggunaan yang berbeda. Pada Gambar 11, sistem menampilkan hasil perjalanan dari GB 5 menuju Dekanat Teknik menggunakan mode berjalan kaki, di mana sistem menghasilkan rute alternatif beserta estimasi jarak dan waktu tempuh yang sesuai dengan kondisi jaringan jalan pedestrian di lingkungan kampus.

Selanjutnya pada Gambar 12, ketika menggunakan mode sepeda motor, sistem menghasilkan rute yang lebih efisien dibandingkan mode berjalan kaki dengan memanfaatkan jalur yang lebih luas dan dapat diakses kendaraan roda dua. Rute yang dihasilkan tetap mempertimbangkan struktur jalan kampus sehingga memberikan keseimbangan antara kecepatan dan aksesibilitas.

Pada Gambar 13, penggunaan mode mobil menunjukkan bahwa sistem mengarahkan rute melalui jalan utama yang sesuai untuk kendaraan roda empat. Namun, karena mobil memiliki keterbatasan akses pada jalan kecil, jalur sempit, serta beberapa area internal kampus, sistem menerapkan mekanisme deteksi anomali rute. Dalam implementasinya, apabila hasil pencarian menunjukkan jarak lebih dari 3 kilometer atau rute dianggap terlalu memutar, sistem secara otomatis melakukan fallback ke rute alternatif yang lebih efisien dan realistis. Mekanisme ini memastikan bahwa hasil rute tidak hanya optimal secara matematis, tetapi juga sesuai dengan kondisi nyata di lapangan.

Pada Gambar 14, sistem menampilkan skenario perjalanan sepeda motor dari GB 5 menuju Gerbang Keluar Belakang. Hasil menunjukkan bahwa sistem mampu menyesuaikan rute berdasarkan kondisi akses area tertentu di dalam kampus, sehingga jalur yang dihasilkan tetap valid dan dapat dilalui sesuai dengan aturan jaringan jalan yang tersedia.

Sementara itu, pada Gambar 15 dan Gambar 16, sistem memperlihatkan pengaruh kondisi waktu operasional kampus terhadap proses pencarian rute. Pada hari kerja dan jam operasional, sistem masih dapat melakukan perhitungan rute secara normal dengan mempertimbangkan akses gerbang yang terbuka. Namun, pada kondisi di luar jam operasional maupun pada hari libur, sistem membatasi atau menghentikan proses pencarian rute dan menampilkan informasi bahwa perhitungan tidak dapat dilakukan secara penuh karena adanya pembatasan akses. Hal ini menunjukkan bahwa sistem telah menerapkan logika berbasis waktu untuk menyesuaikan ketersediaan jalur di lingkungan kampus.

Secara keseluruhan, hasil pengujian pada keenam skenario tersebut menunjukkan bahwa sistem mampu menghasilkan rute yang adaptif terhadap

mode transportasi, kondisi jaringan jalan, serta aturan waktu operasional kampus. Selain itu, adanya mekanisme deteksi anomali pada mode mobil serta fallback rute membuktikan bahwa sistem tidak hanya berfokus pada pencarian jalur terpendek secara matematis, tetapi juga mempertimbangkan kondisi nyata di lapangan agar hasil yang diberikan lebih realistis dan dapat digunakan secara efektif oleh pengguna.

BAB IV

PENUTUP

A. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem yang telah dilakukan, dapat disimpulkan bahwa aplikasi Sistem Navigasi dan Penentuan Rute Terpendek di Lingkungan Universitas Bengkulu Berbasis Web berhasil dikembangkan dengan baik.

Penerapan konsep Shortest Path dan algoritma Dijkstra pada sistem navigasi memungkinkan proses pencarian rute dilakukan secara optimal berdasarkan lokasi awal dan lokasi tujuan yang dipilih pengguna. Dengan memanfaatkan layanan OpenRouteService yang terintegrasi dengan data jaringan jalan, sistem dapat menghasilkan rute perjalanan yang sesuai serta menampilkan alternatif jalur yang dapat dipilih oleh pengguna.

Selain itu, sistem juga mampu mempertimbangkan kondisi operasional gerbang kampus melalui fitur simulasi waktu dan pengaturan akses jalur. Fitur ini memberikan nilai tambah karena rute yang dihasilkan tidak hanya mempertimbangkan jarak terpendek, tetapi juga memperhatikan kondisi aktual yang dapat memengaruhi aksesibilitas pengguna dalam lingkungan kampus.

Visualisasi rute menggunakan peta interaktif berbasis Folium memberikan kemudahan bagi pengguna dalam memahami jalur perjalanan yang direkomendasikan. Dengan demikian, sistem yang dikembangkan dapat menjadi solusi yang membantu mahasiswa, dosen, tamu kampus, maupun masyarakat umum dalam memperoleh informasi navigasi yang lebih cepat, akurat, dan mudah dipahami di lingkungan Universitas Bengkulu.

B. Saran

Di masa mendatang, sistem ini juga dapat dikembangkan menggunakan algoritma pencarian rute lainnya, seperti A^* (*A-Star Algorithm*) atau algoritma optimasi jalur yang lebih kompleks, sehingga dapat dilakukan perbandingan performa untuk memperoleh hasil navigasi yang lebih optimal. Dengan pengembangan dan penyempurnaan yang berkelanjutan, sistem navigasi kampus ini diharapkan dapat menjadi sarana informasi yang lebih lengkap dan mendukung mobilitas civitas akademika Universitas Bengkulu secara efektif.

DAFTAR PUSTAKA

- Bansal, V. K. (2023). *A Road-Based 3D Navigation System in GIS: A Case Study of an Institute Campus*. International Journal of Applied Geospatial
- Bunaen, M. C., Pratiwi, H., & Riti, Y. F. (2022). Penerapan algoritma dijkstra untuk menentukan rute terpendek dari pusat kota surabaya ke tempat bersejarah. *Jurnal Teknologi Dan Sistem Informasi Bisnis-JTEKSIS*, 4(1), 213-223.
- Garingging, C. T. S., Gaol, S. L., Lubis, M. A., Sianturi, F. T., Sembiring, B. M., & Sipayung, S. P. (2025). Implementasi Algoritma Dijkstra dalam Menentukan Rute Terpendek dari Unika St. Thomas Medan ke Lapangan Merdeka. *Jurnal Minfo Polgan*, 14(1), 789-800.
- Giraud, T. (2022). *OSRM: Interface Between R and the OpenStreetMap-Based Routing Service OSRM*. Journal of Open Source Software.
- Paidi, S. S. R. M. R., et al. (2023). *Virtual Ally: A Tableau-Based Campus Navigation System*. International Journal of Information Technology and Computer Engineering.
- Taddesse, A. D., & Tofu, T. K. (2025). *A Novel Approach for Developing Web GIS-Based Navigation System*. International Journal of Soft Computing and Engineering